SET Important Questions and Answers for MSE-1 Unit1 and Unit 2

Questions:

1. Differentiate between functional and non functional requirement by elaborating types of non functional requirement
2. explain agile method in detail
3. explain agile development techniques in detail
4. Explain scrum sprint cycle
5. What is requirement and mention the stake holders of requirement
6. requirement elicitation
7. Extreme programming

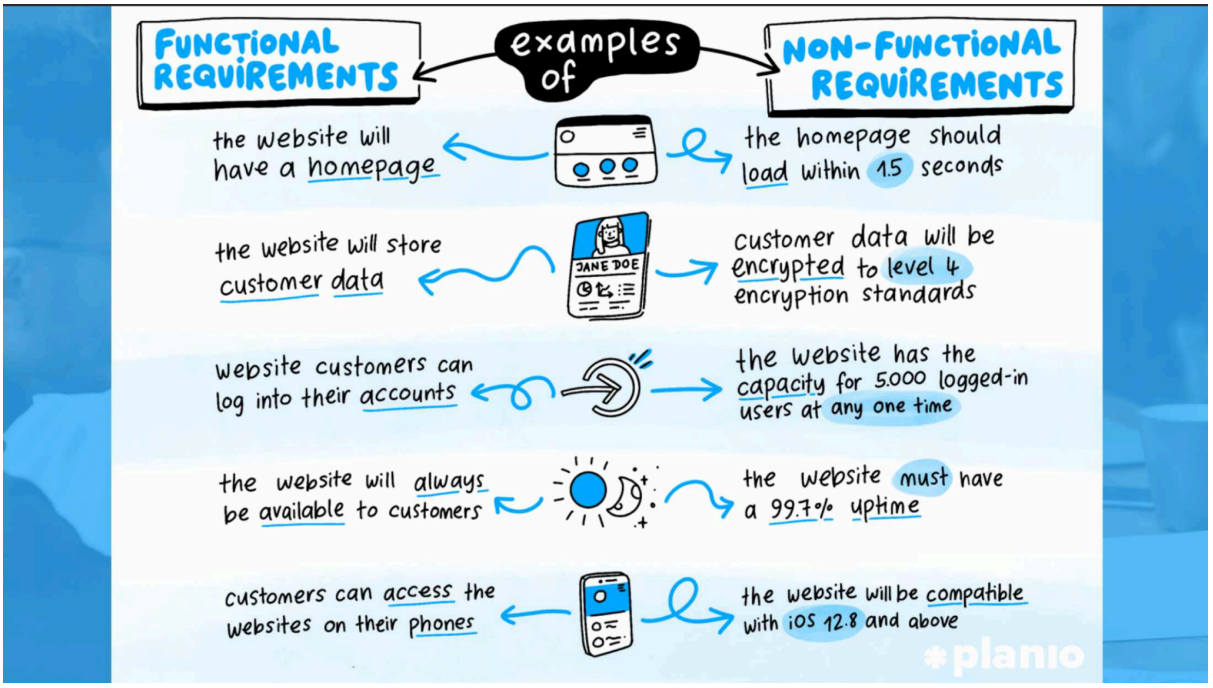
Answers:

1.Differentiate between functional and non functional requirement by elaborating types of non functional requirement

Difference Between Functional and Non-Functional Requirements

Basis	Functional Requirements	Non-Functional Requirements
Definition	Describe **what the system should do**	Describe **how the system performs** its functions
Focus	Services, features, system behavior	Quality attributes and constraints
Scope	Feature-specific	System-wide
Nature	Functional behavior of system	Performance and operational constraints
Measurement	Usually tested by checking correct output	Measured using metrics (time, speed, reliability, etc.)
Example	System shall generate monthly report	Report shall be generated within 2 seconds

Impact	Define system functionality	Affect system architecture and design



Elaboration

1 Functional Requirements

Functional requirements specify the **services or functions** that the system must provide.
They describe:

- Inputs the system accepts
- Processing it performs
- Outputs it produces
- How it behaves in particular situations

They are often written using "**The system shall…**".

Examples:

- The system shall allow users to log in.
- The system shall generate daily appointment lists.
- The system shall store customer data.

These requirements define the **core working of the system**.

2️⃣Non-Functional Requirements (NFR)

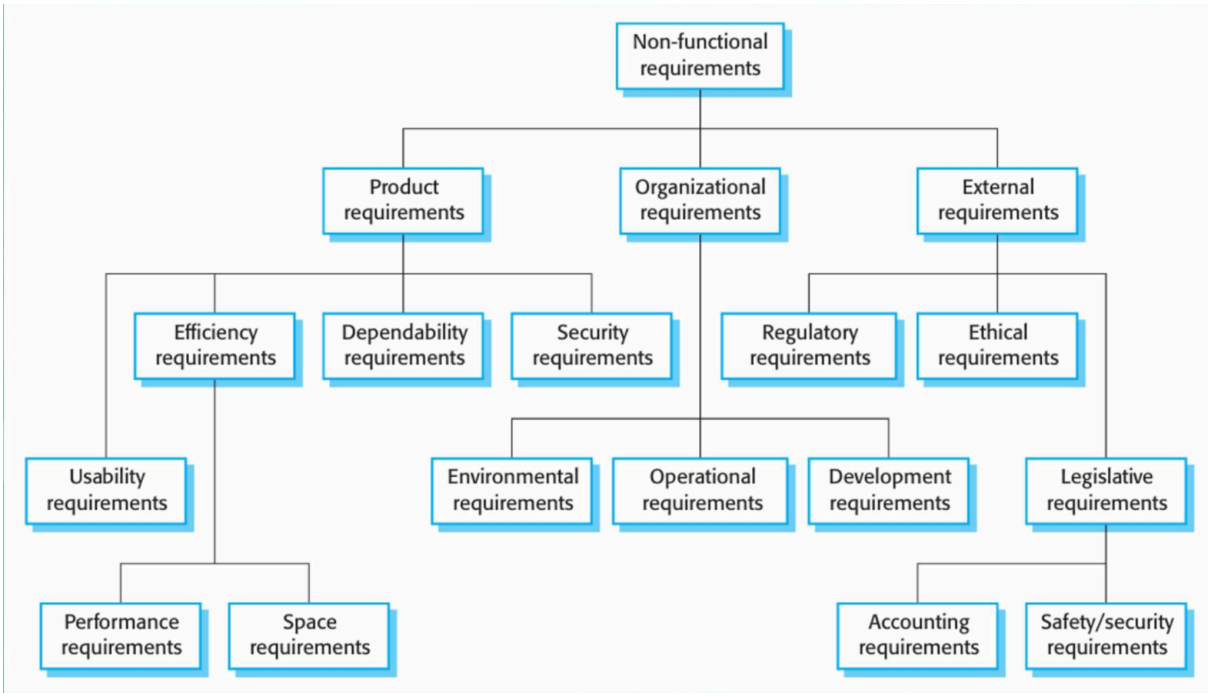
Non-functional requirements define the **quality attributes and constraints** under which the system operates.

They specify:

- Performance
- Reliability
- Security
- Usability
- Standards and legal constraints

If non-functional requirements are not satisfied, even if all functional requirements work, the system may still fail.

Types of Non-Functional Requirements



1️⃣Product Requirements

These specify how the delivered product must behave.

Types:

- **Performance Requirements** – response time, speed

- **Efficiency Requirements** – memory usage, CPU utilization
- **Reliability Requirements** – system uptime, failure rate
- **Security Requirements** – data protection, authentication
- **Usability Requirements** – ease of use

Examples:

- System shall respond within 2 seconds.
- System shall be available 99.9% of the time.
- Data must be encrypted.

2️⃣ Organizational Requirements

These arise from organizational policies or procedures.

Examples:

- System must be developed using Java.
- Development must follow ISO standards.
- Daily backups must be performed.
- Users must authenticate using official ID.

These are constraints imposed by the organization.

3️⃣ External Requirements

These come from factors outside the system.

Types:

- Legal requirements
- Regulatory requirements
- Ethical requirements
- Interoperability requirements

Examples:

- Patient data must comply with privacy laws.
- Financial records must be stored for 7 years.
- System must follow government regulations.

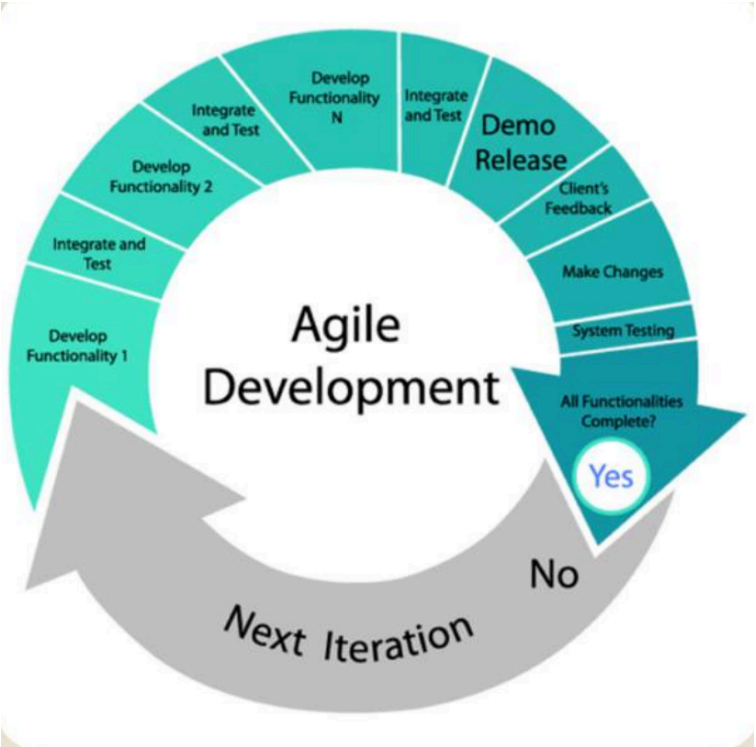
Conclusion

- **Functional requirements** define the system's functionality.
- **Non-functional requirements** define system quality and constraints.

- NFRs are classified into:
    - Product requirements
    - Organizational requirements
    - External requirements

Non-functional requirements often influence the overall system architecture.

2.explain agile method in detail



1️⃣Introduction

Agile is a **modern software development approach** that emphasizes:

- Flexibility
- Customer collaboration
- Iterative development
- Rapid delivery of working software

It was formally introduced through the **Agile Manifesto (2001)**.

Unlike traditional models (like Waterfall), Agile accepts that **requirements change frequently**, and it adapts accordingly.

2️⃣What is Agile?

Agile is an **iterative and incremental development methodology** in which software is developed in small cycles called **iterations or sprints**.

Each iteration:

- Produces a working version of software
- Is tested
- Is reviewed by customers
- Is improved in the next iteration

So instead of delivering the full product at the end, Agile delivers it **step-by-step**.

3️⃣ Agile Manifesto (Core Values)

Agile is based on four main values:

1. Individuals and interactions over processes and tools
2. Working software over comprehensive documentation
3. Customer collaboration over contract negotiation
4. Responding to change over following a plan

This means Agile focuses more on **practical delivery and adaptability** rather than rigid documentation.

4️⃣ Key Characteristics of Agile

According to your notes :

- Software is developed in **small, manageable increments**
- User feedback is collected early and frequently
- Requirements evolve during development
- Changes are easier to implement
- Early working versions are delivered

5️⃣ Agile Development Process

Agile generally follows these steps:

1. Requirement Gathering

High-level requirements are collected (not detailed at the start).

2. Planning

Features are prioritized and divided into small tasks.

3. Iteration (Sprint)

- Design
- Coding
- Testing
- Integration

Each sprint usually lasts **2–4 weeks**.

4. Review

Working software is demonstrated to the customer.

5. Feedback & Improvement

Changes are incorporated in the next sprint.

This cycle repeats until the final product is complete.

6 Advantages of Agile

✔ Early delivery of working software
✔ Handles changing requirements easily
✔ Customer involvement throughout development
✔ Reduced risk
✔ Improved quality through continuous testing
✔ Faster feedback

7 Disadvantages of Agile

✖ Requires active customer involvement
✖ Less emphasis on documentation
✖ Difficult to estimate total cost initially
✖ Not suitable for very large or highly regulated projects

8 Popular Agile Models

Some commonly used Agile frameworks:

- Scrum
- Extreme Programming (XP)
- Kanban

- Lean

Among these, **Scrum** is the most widely used.

9️⃣ Agile vs Waterfall (Important Comparison)

Waterfall	Agile
Linear process	Iterative process
Requirements fixed at start	Requirements evolve
Testing at end	Testing in every iteration
Late delivery	Early & continuous delivery
Low customer involvement	High customer involvement

🔟 When to Use Agile?

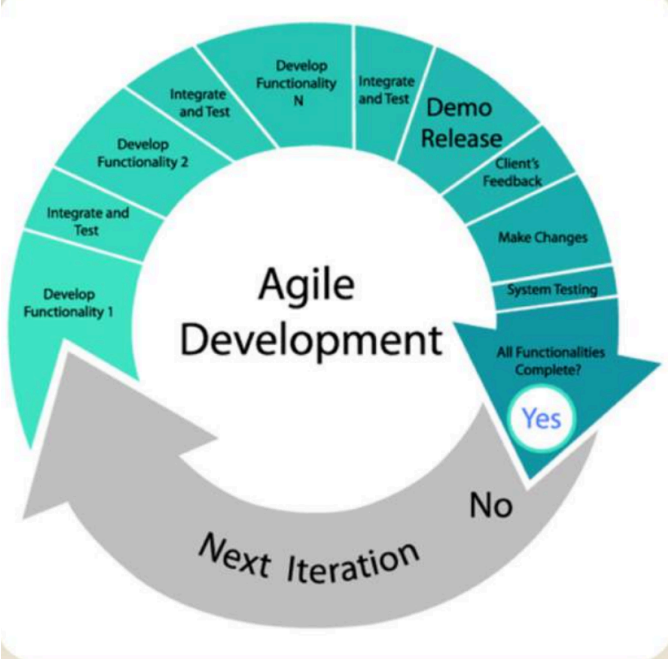
Agile is best suited when:

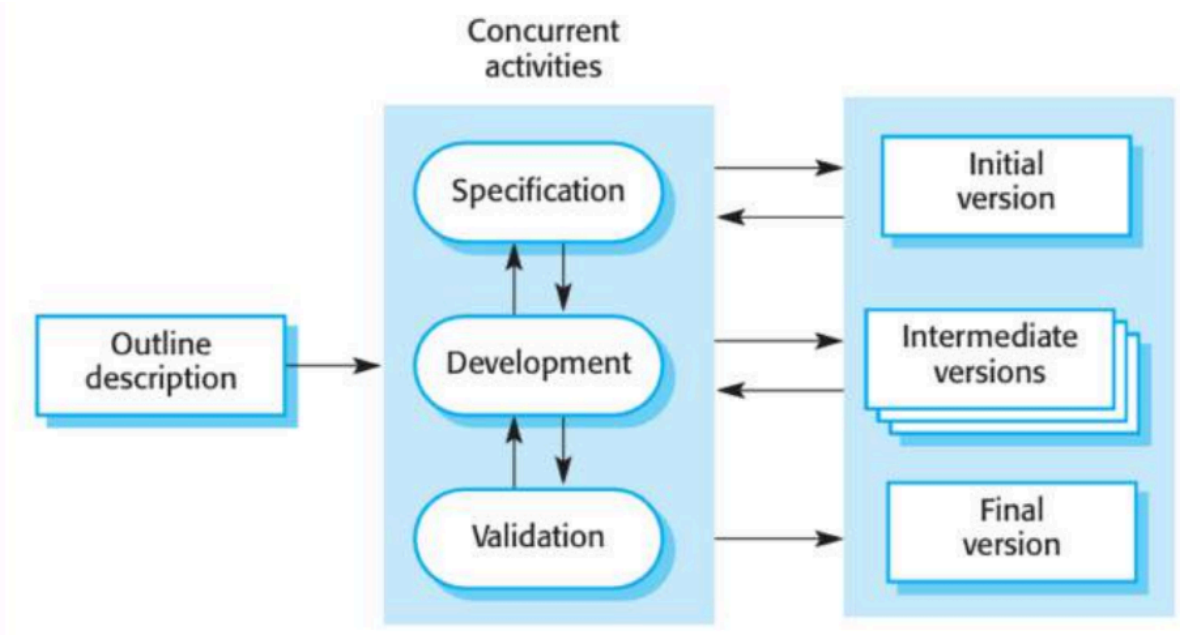
- Requirements are unclear or changing
- Customer feedback is important
- Quick product delivery is required
- Project is small to medium sized

Conclusion

Agile is a **flexible, customer-centered, iterative development method** that delivers software in small increments and adapts to change quickly. It improves customer satisfaction and reduces project risk by continuous testing and feedback.

3.explain agile development techniques in detail





Agile Development Techniques

Agile development techniques are practical practices used to implement Agile principles such as **incremental delivery, continuous feedback, adaptability, and collaboration**.

1️⃣Iterative and Incremental Development

Agile develops software in **small increments (iterations/sprints)** instead of building everything at once.

How it works:

- Requirements are divided into small features.
- Each iteration (2–4 weeks) delivers working software.
- Feedback is collected after every iteration.
- Next version improves or extends functionality.

Benefits:

✔ Early delivery
✔ Easier error detection
✔ Better risk control

2️⃣ User Stories

User stories are short, simple descriptions of features from the user's perspective.

Format:

> As a [user], I want [feature], so that [benefit].

Example:

> As a student, I want to download my result, so that I can check my marks.

Purpose:

- Capture requirements clearly
- Focus on user needs
- Easy to modify

3️⃣ Continuous Customer Involvement

Agile encourages **regular interaction with customers**.

Techniques:

- Sprint reviews
- Feedback meetings
- Prototype demonstrations

Advantage:

Ensures software meets real-world needs.

4️⃣ Continuous Integration (CI)

Continuous Integration means integrating code changes frequently into a shared repository.

Process:

- Developers commit code daily.
- Automated builds and tests are run.
- Errors are detected early.

Benefit:

✔ Reduces integration problems
✔ Improves code quality

5️⃣ Test-Driven Development (TDD)

In TDD, **tests are written before writing actual code**.

Steps:

1. Write a test case.
2. Write minimal code to pass the test.
3. Refactor the code.

Advantages:

✔ Improves quality
✔ Reduces bugs
✔ Ensures requirement correctness

6️⃣ Pair Programming

Two developers work together at one workstation.

- One writes code (Driver).
- Other reviews and suggests improvements (Navigator).

Benefits:

✔ Fewer errors
✔ Knowledge sharing
✔ Better code quality

7️⃣ Refactoring

Refactoring means improving the internal structure of code **without changing its functionality**.

Purpose:

- Remove code duplication
- Improve readability
- Improve maintainability

8️⃣ Daily Stand-up Meetings

Short (10–15 min) daily meetings.

Each member answers:

- What did I do yesterday?
- What will I do today?
- Any obstacles?

Purpose:

✔ Improves communication
✔ Identifies issues quickly

9️⃣ Backlog Prioritization

All features are stored in a **product backlog**.

- Features are prioritized based on importance.
- High-value features are developed first.

This ensures maximum business value delivery.

🔟 Sprint Planning & Review

Sprint Planning:

- Select tasks from backlog.
- Define sprint goal.

Sprint Review:

- Demonstrate working software.
- Collect feedback.

1️⃣1️⃣ Simple Design Principle

Agile follows:

> Build only what is necessary now.

Avoid over-designing.

1️⃣2️⃣ Agile Frameworks Using These Techniques

Common Agile frameworks:

- Scrum
- Extreme Programming (XP)
- Kanban

Extreme Programming (XP) especially emphasizes:

- TDD
- Pair Programming
- Refactoring
- Continuous Integration

Advantages of Agile Techniques

✔ Faster delivery
✔ High flexibility
✔ Better customer satisfaction
✔ Early bug detection
✔ Reduced project risk

Limitations

✖ Requires active stakeholder involvement
✖ Difficult in large, distributed teams
✖ Less documentation

Conclusion

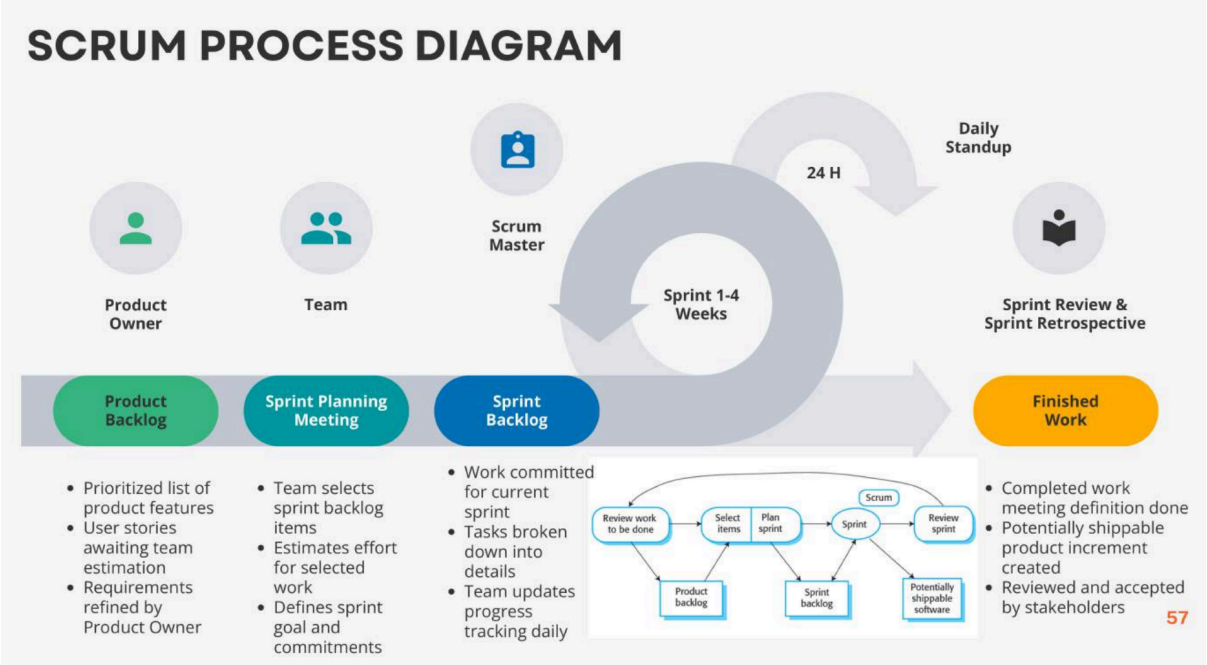
Agile development techniques focus on **incremental development, continuous testing, collaboration, and adaptability**. These techniques help in delivering high-quality software quickly while accommodating changing requirements.

4.Explain scrum sprint cycle

Scrum Sprint Cycle

Scrum is an Agile framework where development happens in fixed-length iterations called **Sprints** (usually 2–4 weeks).
Each sprint delivers a **working, usable increment** of the product.

Scrum Sprint Cycle Overview



1Product Backlog (Before Sprint Starts)

- A list of all features, improvements, and bug fixes.
- Managed by the **Product Owner**.
- Items are written as **user stories**.
- Prioritized based on business value.

2Sprint Planning

This meeting happens at the start of every sprint.

Objectives:

- Select backlog items for the sprint.
- Define the **Sprint Goal**.
- Break user stories into smaller tasks.

Output:

- **Sprint Backlog** (list of tasks to complete in this sprint).

3️⃣ Sprint Execution (Development Phase)

This is the main working period (2–4 weeks).

Activities include:

- Designing
- Coding
- Testing
- Integration

At the end of the sprint, a **working software increment** must be ready.

Important Rule:
No changes are allowed during the sprint unless agreed by the team.

4️⃣ Daily Scrum (Daily Stand-up Meeting)

A short meeting (15 minutes daily).

Each team member answers:

- What did I do yesterday?
- What will I do today?
- Any obstacles?

Purpose:

- Improve communication
- Identify blockers quickly
- Keep sprint on track

5️⃣ Sprint Review

Held at the end of the sprint.

Purpose:

- Demonstrate working software to stakeholders.
- Collect feedback.
- Update product backlog if needed.

Customer feedback may result in:

- New features
- Modifications
- Priority changes

6️⃣ Sprint Retrospective

Internal team meeting after the review.

Discussion:

- What went well?
- What went wrong?
- How can we improve next sprint?

Focus is on **process improvement**.

7️⃣ Start Next Sprint

The cycle repeats:

- Updated backlog
- New sprint planning
- New increment

This continues until the product is complete.

Roles in Scrum Sprint Cycle

Role	Responsibility
Product Owner	Manages product backlog

Scrum Master	Facilitates process, removes obstacles
Development Team	Builds the product

Key Characteristics of Scrum Sprint Cycle

✔ Time-boxed iteration (fixed duration)
✔ Working product at end of each sprint
✔ Continuous feedback
✔ Adaptability to change
✔ Team collaboration

Advantages

- Early delivery of usable software
- Quick adaptation to requirement changes
- Improved product quality
- Reduced project risk

Conclusion

The Scrum Sprint Cycle is a **structured, iterative process** where development occurs in short, time-boxed sprints.
Each sprint results in a working product increment and continuous improvement through review and retrospective meetings.

5.What is requirement and mention the stake holders of requirement

1. What is a Requirement?

A **requirement** is a condition or capability that a system must satisfy to meet the needs of users or stakeholders.

In software engineering, a requirement describes:

- What the system should do
- How the system should behave
- Constraints under which the system operates

According to requirements engineering principles, requirements may include:

- Functional needs
- Non-functional constraints
- Business rules
- External regulations

Types of Requirements (Brief)

1. **Functional Requirements** – Services the system must provide.
2. **Non-Functional Requirements** – Quality attributes and constraints.
3. **Domain Requirements** – Requirements derived from application domain.

2. Stakeholders of Requirements

A **stakeholder** is any person or organization that has an interest in the system or is affected by it.

Stakeholders play a key role in requirements gathering and validation.

Main Stakeholders in Requirements Engineering

1End Users

- People who directly use the system.
- Provide practical requirements.
- Example: Students using an online result system.

2Customers / Clients

- The person or organization that orders and funds the system.
- Define business goals.
- Approve final requirements.

3System Analysts

- Gather, analyze, and document requirements.
- Act as a bridge between users and developers.

4Developers / Engineers

- Design and implement the system.
- Provide technical feasibility input.

5Project Manager

- Plans, schedules, and monitors project progress.
- Ensures requirements align with project constraints.

6Domain Experts

- Provide knowledge about the specific business domain.
- Ensure domain-specific rules are correctly captured.

7Regulatory Authorities

- Ensure compliance with legal and regulatory standards.
- Example: Data protection authorities.

8Maintenance Team

- Responsible for system updates and maintenance.
- Concerned with system sustainability.

Why Stakeholders Are Important

- They provide different perspectives.
- Help identify complete and correct requirements.
- Reduce risk of misunderstandings.
- Improve system acceptance.

Conclusion

A **requirement** is a documented need or condition that a system must fulfill. Stakeholders are individuals or organizations who influence or are affected by the system, and they play a vital role in identifying, validating, and managing requirements.

6.requirement elicitation

1. Definition

Requirement elicitation is the process of gathering, discovering, and identifying the needs and expectations of stakeholders for a software system.

It is the first major activity in **Requirements Engineering**.

In simple words:

> It is the process of collecting requirements from stakeholders.

2. Why Requirement Elicitation is Important

- Stakeholders may not clearly express their needs.
- Different stakeholders may have conflicting requirements.
- Some requirements may be hidden or assumed.
- Helps avoid project failure due to unclear requirements.

3. Activities in Requirement Elicitation

1. Understanding the problem domain
2. Identifying stakeholders
3. Gathering requirements
4. Resolving conflicts
5. Documenting initial requirements

4.Techniques of Requirement Elicitation

1. Interviews

- Direct discussion with stakeholders.
- Can be structured or unstructured.
- Helps gather detailed information.

Advantages:

✔ Direct clarification
✔ In-depth understanding

2. Questionnaires / Surveys

- Written set of questions.
- Used when many stakeholders are involved.

Advantages:

✔ Saves time
✔ Covers large audience

3. Observation

- Analyst observes users performing their tasks.
- Identifies real workflow and hidden requirements.

Useful when:

Users cannot clearly explain their work process.

4. Workshops

- Group discussion sessions.
- Multiple stakeholders participate.
- Helps resolve conflicts quickly.

5. Brainstorming

- Creative idea generation.
- Used to explore new features or solutions.

6. Prototyping

- Developing a preliminary version of the system.
- Stakeholders interact with it.
- Helps clarify unclear requirements.

Types:

- Throwaway prototype
- Evolutionary prototype

7. Document Analysis

- Study existing documents.
- Review current system manuals, reports, forms.

5. Problems in Requirement Elicitation

- Stakeholders may not know what they want.
- Conflicting requirements.
- Communication gap between users and developers.
- Changing requirements.

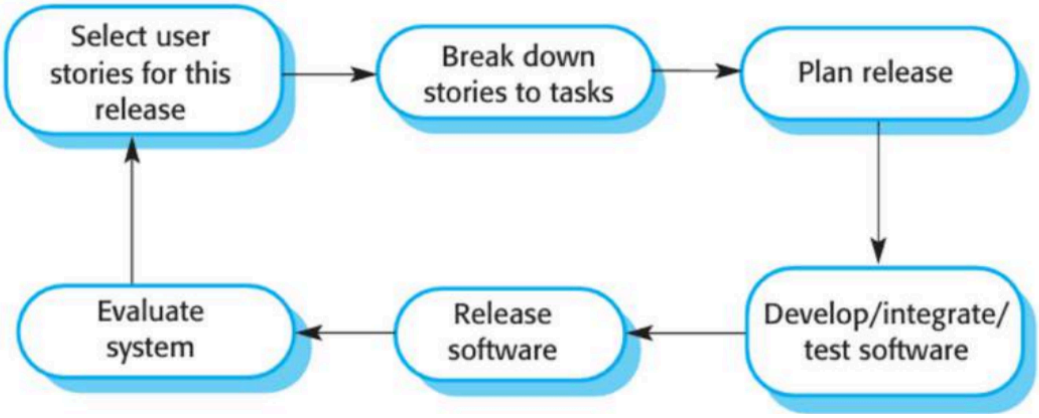
6. Outcome of Requirement Elicitation

- List of gathered requirements
- Initial requirement document
- Better understanding of system scope

Conclusion

Requirement elicitation is the **foundation of software development**.
It ensures that the system is built according to stakeholder needs by using techniques like interviews, observation, workshops, and prototyping.

7.Extreme programming



1. Introduction

Extreme Programming (XP) is an Agile software development methodology that focuses on:

- Improving software quality
- Responding quickly to changing requirements
- Enhancing teamwork and customer satisfaction

It was developed by **Kent Beck** in the late 1990s.

XP is called "extreme" because it takes Agile principles to an extreme level — especially in testing, customer involvement, and collaboration.

2. Objectives of Extreme Programming

- Deliver high-quality software
- Improve customer satisfaction
- Reduce project risk
- Handle changing requirements effectively

3. Core Values of XP

XP is based on five key values:

1. **Communication** – Continuous interaction among team members

2. **Simplicity** – Build only what is needed
3. **Feedback** – Frequent testing and customer review
4. **Courage** – Ability to make changes
5. **Respect** – Mutual trust among team members

4. XP Development Cycle

XP follows short development cycles (iterations), usually 1–2 weeks.

The cycle includes:

1. Planning
2. Design
3. Coding
4. Testing
5. Release

Each iteration produces a working version of the software.

5. Key Practices in Extreme Programming

XP includes several important development practices:

1. Planning Game
- Customer and developers decide features together.
- Features are written as **user stories**.
- Priorities are set for each iteration.

2. Small Releases
- Software is released in small, frequent updates.
- Each release adds new functionality.

3. Test-Driven Development (TDD)
- Tests are written before writing actual code.
- Code is written only to pass the test.
- Improves quality and reduces bugs.

4. Pair Programming
- Two programmers work together at one computer.
- One writes code, the other reviews.
- Improves code quality and knowledge sharing.

5. Continuous Integration
- Code is integrated and tested frequently.
- Errors are detected early.

6. Refactoring
- Improve code structure without changing functionality.
- Removes duplication and improves readability.

7. **Simple Design**
   - Keep the design as simple as possible.
   - Avoid unnecessary complexity.
8. **On-site Customer**
   - Customer is available full-time with the team.
   - Clarifies requirements instantly.

6. Advantages of Extreme Programming

✔ High-quality software
✔ Early detection of errors
✔ Flexible to changing requirements
✔ Strong team collaboration
✔ Continuous customer feedback

7. Disadvantages of XP

✖ Requires active customer involvement
✖ Not suitable for large teams
✖ Less documentation
✖ Difficult in distributed environments

8. XP vs Scrum (Brief Difference)

Scrum	Extreme Programming
Focus on project management	Focus on engineering practices
Sprint duration 2–4 weeks	Iteration 1–2 weeks
No strict coding practices	Strong coding practices (TDD, Pair Programming)

Conclusion

Extreme Programming (XP) is an Agile methodology that emphasizes **testing, collaboration, simplicity, and continuous improvement**. It is best suited for small to medium-sized projects where requirements frequently change.